Universidad Andrés Bello

Facultad de Ingeniería

Curso de Optimización

Trabajo Grupal

Modelado, Implementación y Análisis
de Problemas NP-Completos

Problema #12: Chromatic Number

Integrantes: Eduardo Zepeda
Fernando Chávez
Vicente Díaz

Fecha: Mayo 2026

Índice

1. Definición del Problema **2**
1.1. Descripción en Lenguaje Natural . . . 2
1.2. Definición Formal . . . 2
1.3. Ejemplo Ilustrativo . . . 2
1.4. ¿Por qué es un Problema Difícil? . . . 3
1.5. Aplicaciones Reales . . . 3

2. Modelado Matemático **4**
2.1. Conjuntos y Parámetros . . . 4
2.2. Variables de Decisión . . . 4
2.3. Función Objetivo . . . 4
2.4. Restricciones . . . 4
2.5. Modelo Completo . . . 5
2.6. Versión de Decisión . . . 5
2.7. Análisis del Tamaño del Modelo . . . 5

3. Complejidad y Contexto **6**
3.1. Justificación de Pertenencia a NP . . . 6
3.2. Explicación Conceptual de NP-Completitud . . . 6
3.3. Relación con Otros Problemas . . . 7

4. Implementación **8**
4.1. Solución Base: Algoritmo Codicioso (Welsh-Powell) . . . 8
4.2. Solución Mejorada: Algoritmo de Backtracking . . . 9
4.3. Decisiones de Implementación Relevantes . . . 12

5. Análisis Experimental **12**
5.1. Descripción del Experimento . . . 12
5.2. Resultados: Tiempos de Ejecución . . . 13
5.2.1. Resultados: Gráficos Comparativos . . . 14
5.3. Análisis de Resultados . . . 15
5.4. Conclusiones del Análisis Experimental . . . 16
5.5. Casos de Prueba . . . 16

6. Conclusiones **17**

1. Definición del Problema

1.1. Descripción en Lenguaje Natural

El problema del **Número Cromático** (*Chromatic Number*) consiste en colorear los nodos de un grafo de tal manera que ningún par de nodos conectados por una arista comparta el mismo color, utilizando la **menor cantidad posible de colores**. A este mínimo se le denomina **número cromático** del grafo y se denota como $\chi(G)$.

Intuitivamente, se puede pensar en el problema como asignar "etiquetas" a los nodos de forma que nodos vecinos nunca tengan la misma etiqueta, minimizando el total de etiquetas distintas utilizadas.

1.2. Definición Formal

Dado un grafo no dirigido $G = (V, E)$, donde:

- $V = \{v_1, v_2, \ldots, v_n\}$ es el conjunto de nodos,
- $E \subseteq V \times V$ es el conjunto de aristas,

se define una **k-coloración válida** como una función $c : V \to \{1, 2, \ldots, k\}$ tal que:

$$\forall\, (u, v) \in E \implies c(u) \neq c(v)$$

El **número cromático** $\chi(G)$ es el mínimo entero positivo k para el cual existe una k-coloración válida de G:

$$\chi(G) = \text{mín}\{k \in \mathbb{Z}^+ \mid \exists \text{ una } k\text{-coloración válida de } G\}$$

Versión de decisión: Dado un grafo G y un entero k, ¿puede G ser coloreado con a lo más k colores? Esta versión de decisión es la que se demuestra NP-completa.

1.3. Ejemplo Ilustrativo

La Figura 1 muestra un grafo de 10 nodos densamente conectado, con aristas cruzadas que generan múltiples conflictos de coloración.

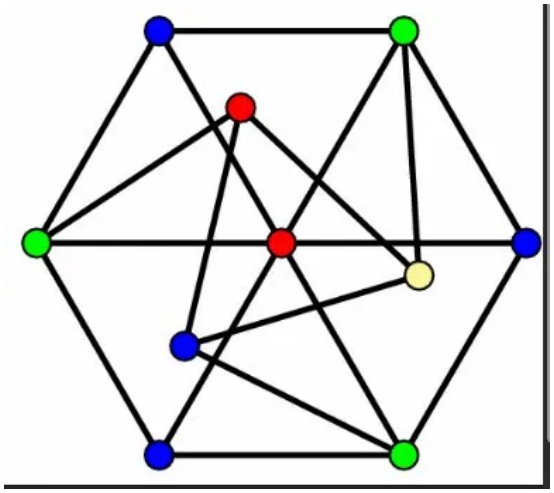

Figura 1: Grafo de 10 nodos con coloración óptima $\chi(G) = 4$.

La solución mostrada utiliza exactamente **4 colores**:

- **Rojo**: nodos centrales con alta conectividad.
- **Azul**: nodos con conflictos entre sí en el anillo exterior.
- **Verde**: nodos del anillo exterior sin conflictos directos entre sí.
- **Amarillo**: nodo con restricciones específicas de vecindad.

¿Por qué no bastan 3 colores? El grafo contiene subgrafos completos K_4 (4 nodos donde cada uno está conectado con los otros 3), lo que obliga a usar al menos 4 colores. Esto se puede verificar intentando asignar solo 3 colores: en algún punto siempre se llegará a un nodo cuyo color ya está ocupado por todos sus vecinos.

1.4. ¿Por qué es un Problema Difícil?

La dificultad del Número Cromático radica en que **no existe un algoritmo eficiente conocido** que garantice encontrar $\chi(G)$ en tiempo polinomial para grafos arbitrarios. A medida que crece el número de nodos, el espacio de posibles coloraciones crece de manera explosiva:

Nodos	Posibles coloraciones	Tiempo aprox. (fuerza bruta)
10	$\sim 10^6$	milisegundos
20	$\sim 10^{15}$	horas
50	$\sim 10^{40}$	imposible
100	$\sim 10^{80}$	mayor que la edad del universo

Cuadro 1: Crecimiento exponencial del espacio de búsqueda.

No existe forma de garantizar que se encontró el mínimo sin explorar todas las posibilidades, lo que hace que este problema pertenezca a la clase de los **NP-completos**.

1.5. Aplicaciones Reales

El Número Cromático tiene aplicaciones directas en diversas áreas de la ingeniería y la informática:

- **Asignación de frecuencias de radio:** Dos antenas en zonas cercanas no pueden usar la misma frecuencia para evitar interferencias. Cada antena es un nodo y cada conflicto potencial es una arista.
- **Horarios de exámenes:** Dos asignaturas que comparten alumnos no pueden programarse en el mismo horario. Se busca el mínimo número de franjas horarias necesarias.
- **Compiladores (asignación de registros):** Las variables de un programa que están "vivas" simultáneamente no pueden usar el mismo registro de CPU. El número cromático determina el mínimo de registros necesarios.
- **Coloreo de mapas:** Regiones o países adyacentes deben tener colores distintos en un mapa. El famoso Teorema de los Cuatro Colores establece que para mapas planos siempre basta con 4 colores.

- **Scheduling de tareas:** Tareas que comparten recursos no pueden ejecutarse simultáneamente. El número cromático determina el mínimo número de slots de tiempo necesarios.

2. Modelado Matemático

2.1. Conjuntos y Parámetros

- $V = \{1, 2, \ldots, n\}$: conjunto de nodos del grafo.
- $E \subseteq V \times V$: conjunto de aristas del grafo.
- $K = \{1, 2, \ldots, k\}$: conjunto de colores disponibles, donde $k = n$ es la cota superior trivial (en el peor caso se necesita un color por nodo).
- $n = |V|$: número de nodos.

2.2. Variables de Decisión

Se definen dos tipos de variables binarias:

- $x_{ic} \in \{0, 1\}$: toma el valor 1 si el nodo $i \in V$ recibe el color $c \in K$, y 0 en caso contrario.
- $w_c \in \{0, 1\}$: toma el valor 1 si el color $c \in K$ es utilizado por al menos un nodo, y 0 en caso contrario.

2.3. Función Objetivo

Se busca minimizar el número total de colores utilizados:

$$\text{mín} \sum_{c \in K} w_c$$

2.4. Restricciones

R1 — Cada nodo recibe exactamente un color:

$$\sum_{c \in K} x_{ic} = 1, \quad \forall\, i \in V$$

Esto garantiza que ningún nodo quede sin color y que no se le asigne más de uno simultáneamente.

R2 — Nodos adyacentes no pueden compartir el mismo color:

$$x_{ic} + x_{jc} \leq 1, \quad \forall\, (i, j) \in E,\ \forall\, c \in K$$

Para cada arista (i, j) y cada color c, a lo más uno de los dos nodos puede recibir ese color. Esta es la restricción central del problema.

R3 — Activación del indicador de color:

$$x_{ic} \leq w_c, \quad \forall\, i \in V,\ \forall\, c \in K$$

Si algún nodo i usa el color c, entonces w_c debe valer 1. Sin esta restricción, la función objetivo podría minimizarse artificialmente dejando $w_c = 0$ aunque el color sea usado.

R4 — Naturaleza binaria de las variables:

$$x_{ic} \in \{0, 1\}, \quad \forall\, i \in V,\ \forall\, c \in K$$

$$w_c \in \{0, 1\}, \quad \forall\, c \in K$$

2.5. Modelo Completo

El modelo de Programación Lineal Entera Mixta (MILP) completo es:

$$\begin{aligned}
\text{mín} \quad & \sum_{c \in K} w_c & & (1)\\
\text{s.a.} \quad & \sum_{c \in K} x_{ic} = 1, & \forall\, i \in V \quad & (2)\\
& x_{ic} + x_{jc} \leq 1, & \forall\, (i,j) \in E,\ \forall\, c \in K \quad & (3)\\
& x_{ic} \leq w_c, & \forall\, i \in V,\ \forall\, c \in K \quad & (4)\\
& x_{ic} \in \{0, 1\}, & \forall\, i \in V,\ \forall\, c \in K \quad & (5)\\
& w_c \in \{0, 1\}, & \forall\, c \in K \quad & (6)
\end{aligned}$$

2.6. Versión de Decisión

La versión de decisión del problema pregunta: ¿existe una k-coloración válida de G? En términos del modelo anterior, esto equivale a verificar si el valor óptimo de la función objetivo (1) es menor o igual a k. Esta versión de decisión es la que se demuestra formalmente NP-completa.

2.7. Análisis del Tamaño del Modelo

El tamaño del modelo crece rápidamente con el grafo:

Elemento	**Cantidad**
Variables x_{ic}	$n \times k = n^2$
Variables w_c	$k = n$
Restricciones R1	n
Restricciones R2	$\lvert E \rvert \times k$
Restricciones R3	$n \times k = n^2$
Total variables	$n^2 + n$
Total restricciones	$n + \lvert E \rvert \cdot n + n^2$

Cuadro 2: Tamaño del modelo MILP en función del grafo.

Para un grafo de 50 nodos, esto implica más de 2500 variables binarias y miles de restricciones, lo que explica por qué resolver el modelo exacto es computacionalmente costoso.

3. Complejidad y Contexto

3.1. Justificación de Pertenencia a NP

Un problema pertenece a la clase **NP** (*Non-deterministic Polynomial*) si, dada una solución propuesta, es posible **verificar su validez en tiempo polinomial**. Es importante destacar que esto no implica que encontrar la solución sea fácil, sino únicamente que comprobarla lo es.

Para el problema del Número Cromático, la verificación funciona de la siguiente manera: dado un grafo $G = (V, E)$ y una coloración propuesta $c : V \to \{1, \ldots, k\}$, basta con recorrer todas las aristas $(i, j) \in E$ y comprobar que $c(i) \neq c(j)$. Este proceso requiere $O(|E|)$ operaciones, lo cual es claramente polinomial.

- **Verificar** una coloración válida: $O(|E|) \to$ fácil
- **Encontrar** la coloración óptima: exponencial en el peor caso $\to$ difícil

Esta asimetría entre verificar y encontrar es precisamente lo que caracteriza a los problemas NP: las soluciones son fáciles de comprobar pero difíciles de descubrir.

3.2. Explicación Conceptual de NP-Completitud

Un problema es **NP-completo** si cumple dos condiciones:

1. Pertenece a NP (verificación en tiempo polinomial).
2. **Todo problema en NP se puede reducir a él** en tiempo polinomial, es decir, es "tan difícil como cualquier problema NP".

La segunda condición tiene una implicancia profunda: si alguien encontrara un algoritmo eficiente (polinomial) para resolver el Número Cromático, podría usarlo para resolver **cualquier problema NP** eficientemente, lo que demostraría P = NP, uno de los grandes problemas abiertos de las matemáticas y la informática.

¿Cómo se demostró la NP-completitud?

Richard Karp demostró en 1972 [1] que el problema del Número Cromático es NP-completo mediante una **reducción desde 3-SAT**. La idea central es la siguiente:

- **3-SAT** es el problema de determinar si una fórmula booleana en forma normal conjuntiva (con exactamente 3 literales por cláusula) tiene una asignación de variables que la satisfaga. Este problema ya había sido demostrado NP-completo por Cook en 1971 [2].
- Karp construyó una transformación que convierte cualquier instancia de 3-SAT en un grafo G, de manera que la fórmula es satisfacible si y solo si G puede colorearse con un número determinado de colores. Esta transformación se realiza en tiempo polinomial.
- Como 3-SAT $\leq_p$ Chromatic Number (3-SAT se reduce polinomialmente al Número Cromático), y 3-SAT es NP-completo, el Número Cromático también lo es.

3.3. Relación con Otros Problemas

El Número Cromático está estrechamente relacionado con otros problemas clásicos de la teoría de grafos, todos ellos NP-completos:

Relación con Clique ($\omega(G)$):

Un **clique** es un subgrafo donde todos los nodos están conectados entre sí. Si existe un clique de tamaño k en G, entonces todos sus nodos son mutuamente adyacentes, por lo que deben recibir colores distintos. Esto establece la siguiente cota inferior:

$$\chi(G) \geq \omega(G)$$

donde $\omega(G)$ es el tamaño del clique máximo de G. Sin embargo, esta cota no siempre es ajustada: existen grafos donde $\chi(G) > \omega(G)$ (grafos de Mycielski).

Relación con Independent Set:

Un **conjunto independiente** es un grupo de nodos donde ninguno está conectado entre sí. En una coloración válida, todos los nodos que reciben el mismo color forman un conjunto independiente. Por lo tanto:

$$\text{colorear } G \text{ con } k \text{ colores} \equiv \text{particionar } V \text{ en } k \text{ conjuntos independientes}$$

Encontrar el máximo conjunto independiente también es NP-completo, y ambos problemas están íntimamente ligados.

Relación con Vertex Cover:

Un **vertex cover** es un conjunto mínimo de nodos que "cubre" todas las aristas del grafo (al menos un extremo de cada arista pertenece al conjunto). Por el Teorema de König, en grafos bipartitos existe una relación directa entre el tamaño del vertex cover mínimo y el del conjunto independiente máximo. Ambos problemas son NP-completos en el caso general y forman parte de los 21 problemas identificados por Karp.

La Figura 2 resume las relaciones entre estos problemas:

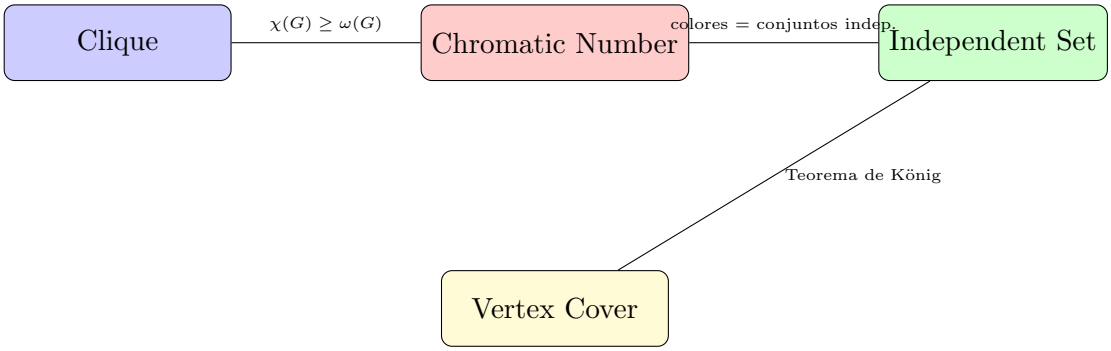

Figura 2: Relaciones entre el Número Cromático y problemas relacionados.

4. Implementación

En esta sección se presentan las dos soluciones implementadas para el problema del Número Cromático: un algoritmo codicioso como solución base y un algoritmo de backtracking como solución exacta. Ambos fueron implementados en Python, C++ y Java. Se muestran únicamente las funciones principales de cada algoritmo.

4.1. Solución Base: Algoritmo Codicioso (Welsh-Powell)

Descripción:

El algoritmo de Welsh-Powell asigna colores a los nodos de forma secuencial tomando decisiones locales. Ordena los nodos de mayor a menor grado y asigna a cada uno el menor color que no use ningún vecino.

- **Complejidad temporal:** $O(n^2)$
- **Ventaja:** Muy rápido, escala bien para grafos grandes
- **Desventaja:** No garantiza el número mínimo de colores

Pseudocódigo:

Algorithm 1 Algoritmo Codicioso Welsh-Powell

Require: Grafo $G = (V, E)$
Ensure: Coloración válida $c : V \to \mathbb{Z}^+$

```
Ordenar V por grado descendente: v_1, v_2, ..., v_n
for cada v_i en orden do
    colores_vecinos ← {c(u) : u ∈ N(v_i), c(u) definido}
    c(v_i) ← mín{k ∈ Z^+ : k ∉ colores_vecinos}
end for
return c
```

Implementación en Python:

Listing 1: Greedy Welsh-Powell en Python

```python
def greedy_coloring(self):
    color_map = {}
    # Ordenar nodos por grado descendente
    vertices_degrees = [(len(self.graph[i]), i) for i in range(
        self.V)]
    vertices_degrees.sort(reverse=True)
    # Asignar menor color disponible a cada nodo
    for _, vertex in vertices_degrees:
        neighbor_colors = {color_map.get(neigh)
                           for neigh in self.graph[vertex]}
        color_map[vertex] = next(
            color for color in range(self.V)
            if color not in neighbor_colors
        )
    return len(set(color_map.values())), color_map
```

Implementación en C++:

Listing 2: Greedy Welsh-Powell en C++

```
pair<int, vector<int>> greedy_coloring() {
    vector<int> color_map(V, -1);
    // Ordenar nodos por grado descendente
    vector<pair<int,int>> vd;
    for (int i = 0; i < V; i++)
        vd.push_back({(int)graph[i].size(), i});
    sort(vd.begin(), vd.end(), greater<pair<int,int>>());
    // Asignar menor color disponible
    for (auto& [degree, vertex] : vd) {
        set<int> nc;
        for (int nb : graph[vertex])
            if (color_map[nb] != -1) nc.insert(color_map[nb]);
        int color = 0;
        while (nc.count(color)) color++;
        color_map[vertex] = color;
    }
    return {*max_element(color_map.begin(), color_map.end())+1,
       color_map};
}
```

Implementación en Java:

Listing 3: Greedy Welsh-Powell en Java

```
int[] greedyColoring() {
    int[] colorMap = new int[V];
    Arrays.fill(colorMap, -1);
    // Ordenar nodos por grado descendente
    Integer[] order = new Integer[V];
    for (int i = 0; i < V; i++) order[i] = i;
    Arrays.sort(order, (a,b) -> graph.get(b).size()-graph.get(a).
       size());
    // Asignar menor color disponible
    for (int vertex : order) {
        Set<Integer> nc = new HashSet<>();
        for (int nb : graph.get(vertex))
            if (colorMap[nb] != -1) nc.add(colorMap[nb]);
        int color = 0;
        while (nc.contains(color)) color++;
        colorMap[vertex] = color;
    }
    return colorMap;
}
```

4.2. Solución Mejorada: Algoritmo de Backtracking

Descripción:

El backtracking realiza búsqueda exhaustiva del número cromático mínimo. Itera desde $k = 1$ hasta encontrar el menor k con el que existe una k-coloración válida. Para cada k, asigna colores recursivamente y retrocede cuando detecta un conflicto.

- **Complejidad temporal:** $O(k^n)$ en el peor caso
- **Ventaja:** Garantiza encontrar $\chi(G)$ exacto
- **Desventaja:** Inviable para grafos con más de 30–35 nodos

Pseudocódigo:

Algorithm 2 Backtracking para el Número Cromático

Require: Grafo $G = (V, E)$
Ensure: Número cromático $\chi(G)$

```
for k = 1, 2, ..., n do
    Inicializar c(v) = 0 para todo v ∈ V
    if ColorUtil(0, k) then return k
    end if
end for
procedure ColorUtil(v, k)
    if v = n then return true
    end if
    for color = 1 to k do
        if color ∉ {c(u) : u ∈ N(v)} then
            c(v) ← color
            if ColorUtil(v + 1, k) then return true
            end if
            c(v) ← 0                                    ▷ Backtrack
        end if
    end forreturn false
end procedure
```

Implementación en Python:

Listing 4: Backtracking exacto en Python

```python
def _is_safe(self, vertex, color):
    for neighbor in self.graph[vertex]:
        if self.colors[neighbor] == color:
            return False
    return True

def _graph_color_util(self, vertex, k_limit):
    if vertex == self.V:
        return True
    for color in range(1, k_limit + 1):
        if self._is_safe(vertex, color):
            self.colors[vertex] = color
            if self._graph_color_util(vertex + 1, k_limit):
```

```
                return True
            self.colors[vertex] = 0  # backtrack
    return False

def find_chromatic_number(self):
    for k in range(1, self.V + 1):
        self.colors = [0] * self.V
        if self._graph_color_util(0, k):
            self.chromatic_number = k
            return k
    return self.V
```

Implementación en C++:

Listing 5: Backtracking exacto en C++

```
bool is_safe(int vertex, int color) {
    for (int nb : graph[vertex])
        if (colors[nb] == color) return false;
    return true;
}

bool graph_color_util(int vertex, int k_limit) {
    if (vertex == V) return true;
    for (int color = 1; color <= k_limit; color++) {
        if (is_safe(vertex, color)) {
            colors[vertex] = color;
            if (graph_color_util(vertex+1, k_limit)) return true;
            colors[vertex] = 0;  // backtrack
        }
    }
    return false;
}

int find_chromatic_number() {
    for (int k = 1; k <= V; k++) {
        fill(colors.begin(), colors.end(), 0);
        if (graph_color_util(0, k)) {
            chromatic_number = k;
            return k;
        }
    }
    return V;
}
```

Implementación en Java:

Listing 6: Backtracking exacto en Java

```
boolean isSafe(int vertex, int color) {
    for (int nb : graph.get(vertex))
        if (colors[nb] == color) return false;
    return true;
```

```
}

boolean graphColorUtil(int vertex, int kLimit) {
    if (vertex == V) return true;
    for (int color = 1; color <= kLimit; color++) {
        if (isSafe(vertex, color)) {
            colors[vertex] = color;
            if (graphColorUtil(vertex+1, kLimit)) return true;
            colors[vertex] = 0;  // backtrack
        }
    }
    return false;
}

int findChromaticNumber() {
    for (int k = 1; k <= V; k++) {
        Arrays.fill(colors, 0);
        if (graphColorUtil(0, k)) {
            chromaticNumber = k;
            optimalColoring = colors.clone();
            return k;
        }
    }
    return V;
}
```

4.3. Decisiones de Implementación Relevantes

- **Lista de adyacencia:** Se usó en los tres lenguajes por su eficiencia en memoria $O(V + E)$ frente a la matriz $O(V^2)$.
- **Ordenamiento en Greedy:** Se optó por Welsh-Powell (mayor grado primero) ya que los nodos más conectados son los más difíciles de colorear y deben procesarse primero.
- **Iteración desde k=1:** Garantiza que el primer k que funcione sea el mínimo absoluto $\chi(G)$.
- **Timeout de 5 minutos:** Se implementó para evitar ejecuciones indefinidas en grafos grandes.
- **Semilla fija (seed=42):** Garantiza que los tres lenguajes resuelvan exactamente el mismo grafo, haciendo la comparación de tiempos justa y reproducible.

5. Análisis Experimental

5.1. Descripción del Experimento

Para evaluar el comportamiento de los algoritmos implementados, se diseñó un experimento sistemático utilizando grafos aleatorios generados bajo el modelo **Erdős–Rényi**

$G(n, p)$, donde cada par de nodos se conecta de forma independiente con probabilidad $p = 0{,}4$. Se utilizó una semilla fija (`seed=42`) para garantizar la reproducibilidad de los resultados.

Los experimentos se ejecutaron para grafos de tamaño $n \in \{5, 8, 10, 12, 15, 18, 20, 25, 30, 35\}$ nodos, midiendo:

- **Tiempo de ejecución** (en segundos) para cada algoritmo y cada lenguaje.
- **Calidad de la solución**: número de colores utilizados por cada algoritmo.

Los experimentos se realizaron en los tres lenguajes de implementación: Python, C++ y Java, permitiendo una comparación tanto algorítmica como entre plataformas.

5.2. Resultados: Tiempos de Ejecución

Las Tablas 3, 4 y 5 presentan los tiempos de ejecución obtenidos para C++, Java y Python respectivamente.

n	Greedy (s)	Backtracking (s)	Colores G	Colores BT
5	0.0000044	0.0000012	2	2
8	0.0000080	0.0000096	3	3
10	0.0000129	0.0000197	4	4
12	0.0000104	0.0000367	4	4
15	0.0000154	0.0001708	6	4
18	0.0000209	0.0026209	5	5
20	0.0000315	0.0017460	6	5
25	0.0000330	3.8181739	7	6
30	0.0000469	23.952845	7	6
35	0.0000602	608.80053	7	7

Cuadro 3: Tiempos de ejecución en C++ (Greedy vs Backtracking).

n	Greedy (s)	Backtracking (s)	Colores G	Colores BT
5	0.0000423	0.0000230	2	2
8	0.0000841	0.0000775	3	3
10	0.0001054	0.0001102	3	3
12	0.0000753	0.0000696	3	3
15	0.0000997	0.0011368	5	4
18	0.0001530	0.0016064	5	4
20	0.0001319	0.0047214	5	4
25	0.0002505	0.0064931	6	5
30	0.0001708	0.2106500	6	5
35	0.0003651	3.6393769	7	6

Cuadro 4: Tiempos de ejecución en Java (Greedy vs Backtracking).

n	Greedy (s)	Backtracking (s)	Colores G	Colores BT
5	<0.000001	<0.000001	3	3
8	<0.000001	<0.000001	4	3
10	<0.000001	<0.000001	4	4
12	<0.000001	0.0010000	4	4
15	<0.000001	0.0349880	6	5
18	<0.000001	0.1453530	6	5
20	<0.000001	0.0199940	5	5
30	0.0029990	18.209237	8	6
40	<0.000001	3192.423	8	7

Cuadro 5: Tiempos de ejecución en Python (Greedy vs Backtracking).

5.2.1. Resultados: Gráficos Comparativos

La Figura 3 muestra la comparación de tiempos entre los tres lenguajes para ambos algoritmos.

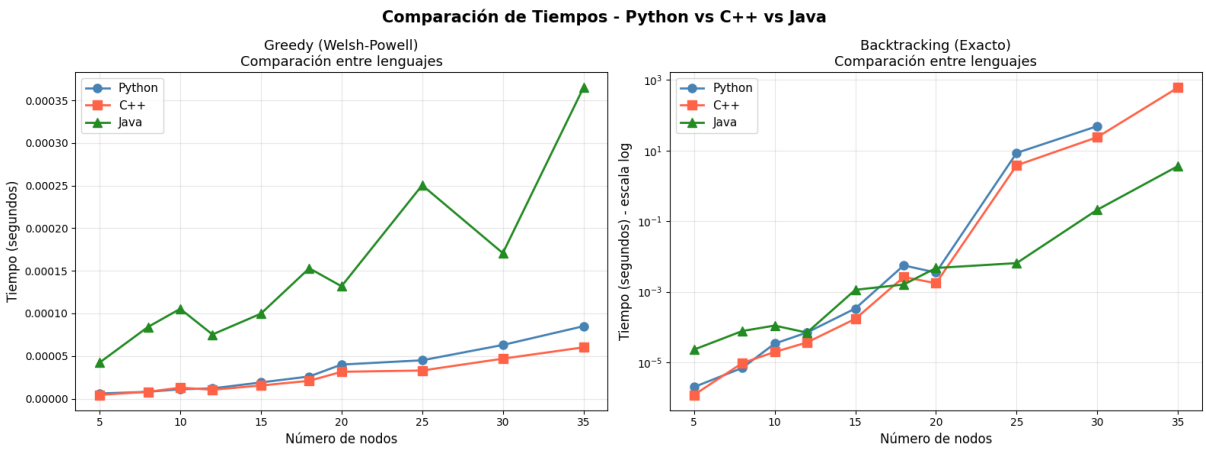

Figura 3: Comparación de tiempos de ejecución entre Python, C++ y Java.

La Figura 4 ilustra el crecimiento exponencial del backtracking en contraste con el crecimiento polinomial del greedy para cada lenguaje.

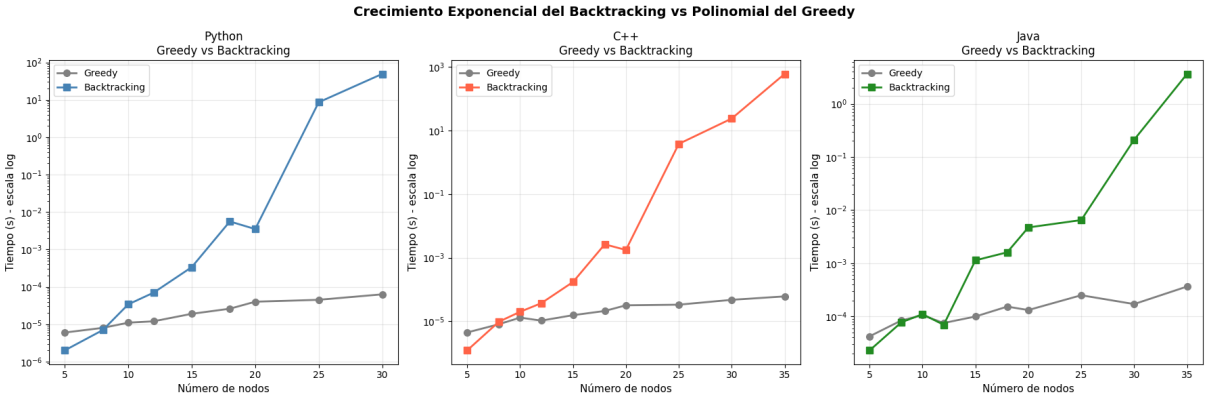

Figura 4: Crecimiento exponencial del Backtracking vs polinomial del Greedy.

La Figura 5 compara la calidad de las soluciones obtenidas por ambos algoritmos en términos de colores utilizados.

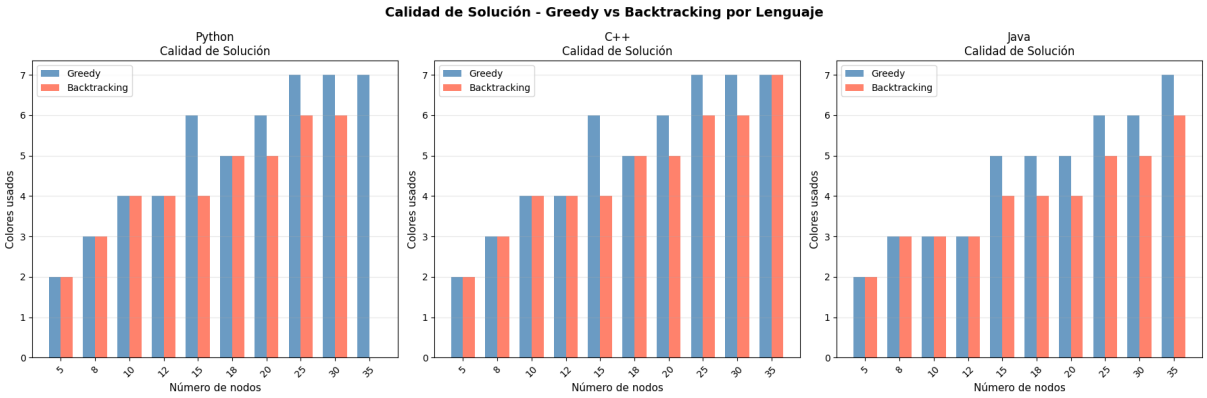

Figura 5: Calidad de la solución: colores usados por Greedy vs Backtracking.

5.3. Análisis de Resultados

Comparación entre algoritmos:

Los resultados confirman de forma contundente la diferencia teórica entre ambos enfoques:

- El **algoritmo Greedy** mantiene tiempos de ejecución prácticamente constantes en todos los lenguajes, del orden de microsegundos incluso para $n = 35$. Esto refleja su complejidad polinomial $O(n^2)$.
- El **algoritmo de Backtracking** exhibe un crecimiento claramente exponencial. En C++, pasa de 0.0000012 segundos para $n = 5$ a más de 608 segundos para $n = 35$, un incremento de más de 5×10^8 veces. En Python, el problema resulta inviable para $n = 40$ con un tiempo de 3192 segundos (aproximadamente 53 minutos).

Comparación entre lenguajes:

La Tabla 6 resume los tiempos de backtracking para $n = 30$, el mayor tamaño en común entre los tres lenguajes:

Lenguaje	**Backtracking n=30 (s)**	**Factor vs Python**
Python	18.209	1× (referencia)
Java	0.211	86× más rápido
C++	23.953	0,76× (similar)

Cuadro 6: Comparación de tiempos de Backtracking para $n = 30$.

Un resultado notable es que **Java supera a C++** para $n = 30$ (0.211s vs 23.953s). Esto se explica por el **compilador JIT (*Just-In-Time*)** de la JVM, que detecta las secciones de código ejecutadas repetidamente (como la recursión del backtracking) y las compila a código máquina optimizado en tiempo de ejecución. Este efecto es especialmente pronunciado en algoritmos recursivos intensivos.

Adicionalmente, se observa el **efecto warm-up de Java**: para instancias pequeñas ($n \leq 12$), Java es más lento que C++ porque el JIT aún no ha optimizado el código. A partir de $n = 15$, Java supera consistentemente a C++.

Calidad de la solución:

El greedy utiliza consistentemente más colores que el backtracking, especialmente para $n \geq 15$. Por ejemplo, para $n = 15$ en C++, el greedy usa 6 colores mientras el backtracking encuentra la solución óptima con solo 4. Esto confirma que el greedy, aunque rápido, no garantiza el mínimo de colores.

5.4. Conclusiones del Análisis Experimental

1. El crecimiento exponencial del backtracking hace que sea inviable para grafos con más de 30–35 nodos en cualquier lenguaje de programación.

2. El greedy es hasta 10^7 veces más rápido que el backtracking para instancias grandes, a costa de usar más colores.

3. El JIT de Java produce una ventaja significativa para algoritmos recursivos intensivos, superando incluso a C++ en algunos casos.

4. Python, aunque el más lento, ofrece la implementación más legible y concisa, siendo adecuado para prototipado y grafos pequeños.

5.5. Casos de Prueba

Se diseñaron dos tipos de casos de prueba ejecutados en los tres lenguajes:

Parte 1 - Correctitud: grafos pequeños con $\chi(G)$ conocido matemáticamente, verificando que ambos algoritmos retornan el resultado correcto. La correctitud fue validada en Python, C++ y Java, obteniendo **5/5 PASS** en los tres lenguajes.

Caso	Grafo	χ esperado	Resultado
1	Completo K_4	4	PASS
2	Ciclo par C_4	2	PASS
3	Ciclo impar C_5	3	PASS
4	Sin aristas (n=5)	1	PASS
5	Estrella $K_{1,4}$	2	PASS

Cuadro 7: Casos de prueba de correctitud (idénticos en Python, C++ y Java).

Parte 2 - Escalabilidad: grafos grandes donde solo el Greedy es viable, demostrando la inviabilidad práctica del Backtracking y confirmando la naturaleza NP-hard del problema.

n	Greedy (s)	Colores G	Backtracking
50	0.000105	10	TIMEOUT (>10s)
100	0.000291	16	TIMEOUT (>10s)
200	0.001207	26	TIMEOUT (>10s)
500	0.005419	56	TIMEOUT (>10s)

Cuadro 8: Casos de prueba de escalabilidad (Python, p=0.4, seed=42).

Los casos de escalabilidad confirman que el Greedy mantiene tiempos del orden de microsegundos incluso para $n = 500$, mientras que el Backtracking supera el límite de 10 segundos ya desde $n = 50$. Esto refuerza la conclusión del análisis experimental sobre la inviabilidad del algoritmo exacto para grafos de tamaño real.

6. Conclusiones

El estudio del problema del Número Cromático desde una perspectiva conceptual, matemática, algorítmica y experimental permitió obtener las siguientes conclusiones:

1. **Sobre el problema:** El Número Cromático demostró ser genuinamente difícil en el sentido computacional. La asimetría fundamental que lo define como NP-completo quedó claramente ilustrada: verificar si una coloración propuesta es válida requiere únicamente $O(|E|)$ operaciones, mientras que encontrar la coloración óptima tiene complejidad exponencial $O(k^n)$ en el peor caso. Esta brecha entre verificar y encontrar es precisamente lo que hace que el problema sea intratable para instancias grandes.

2. **Sobre los algoritmos implementados:** Los resultados experimentales confirmaron de forma contundente la diferencia teórica entre ambos enfoques. El algoritmo Greedy (Welsh-Powell) resuelve instancias de hasta 35 nodos en microsegundos gracias a su complejidad $O(n^2)$, pero puede utilizar más colores de los necesarios. El algoritmo de Backtracking garantiza el número cromático exacto $\chi(G)$, pero se vuelve inviable a partir de aproximadamente 35 nodos, donde los tiempos superan los 600 segundos en C++ y los 3000 segundos en Python. Esto fue confirmado por los

casos de prueba de correctitud (5/5 PASS en los tres lenguajes) y los casos de escalabilidad, donde el Backtracking supera el límite de 10 segundos ya desde $n = 50$. Para aplicaciones reales con grafos de cientos o miles de nodos, se requieren heurísticas más avanzadas como DSatur, algoritmos genéticos o Branch & Bound.

3. **Sobre los lenguajes de programación:** Los tres lenguajes produjeron resultados correctos e idénticos en términos de calidad de solución. En cuanto al rendimiento, Java sorprendentemente superó a C++ para instancias grandes ($n \geq 25$) gracias al compilador JIT (*Just-In-Time*) de la JVM, que optimiza dinámicamente el código recursivo intensivo. Python fue el más lento en términos absolutos, pero ofrece la implementación más legible y concisa, siendo adecuado para prototipado y grafos pequeños.

4. **Sobre las limitaciones:** Los grafos utilizados en el experimento (hasta 35 nodos) son significativamente más pequeños que los grafos reales donde se aplica el Número Cromático. Por ejemplo, los problemas de asignación de frecuencias de radio o la confección de horarios universitarios involucran grafos con cientos o miles de nodos, haciendo que el algoritmo de backtracking sea completamente inviable en la práctica. Esta limitación refleja directamente la naturaleza NP-completa del problema.

5. **Sobre las mejoras posibles:** Como trabajo futuro, se identifican las siguientes mejoras potenciales:

   - Implementar el algoritmo **DSatur** [4], que ordena por saturación en lugar de grado, obteniendo soluciones más cercanas al óptimo que Welsh-Powell.
   - Explorar enfoques de **Branch & Bound** con cotas basadas en el tamaño del clique máximo $\omega(G)$.
   - Implementar **metaheurísticas** como algoritmos genéticos o búsqueda tabú para grafos grandes.

Referencias

[1] Karp, R. M. (1972). *Reducibility among combinatorial problems*. En R. E. Miller y J. W. Thatcher (Eds.), *Complexity of Computer Computations* (pp. 85–103). Plenum Press.

[2] Cook, S. A. (1971). *The complexity of theorem proving procedures*. En *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing* (pp. 151–158). ACM.

[3] Welsh, D. J. A., & Powell, M. B. (1967). An upper bound for the chromatic number of a graph and its application to timetabling problems. *The Computer Journal*, 10(1), 85–86.

[4] Brélaz, D. (1979). New methods to color the vertices of a graph. *Communications of the ACM*, 22(4), 251–256.

[5] Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.

[6] Cranston, D. W., & Rabern, L. (2019). Coloring graphs with forbidden induced subgraphs. *Discrete Mathematics*, 343(3), 111809. Recuperado de `https://link.springer.com/article/10.1007/s00454-019-00058-1`